

13a — Algorithmen modellieren

Vom Problem zur Lösung: Pseudocode und Ablaufdiagramme

HTW Berlin – Wirtschaftsingenieurwesen

SoSe 2026

Agenda

1. Was ist ein Algorithmus?
2. Eigenschaften: was unterscheidet Algorithmus von “Code, der irgendwie läuft”
3. Pseudocode – die Sprache vor der Sprache
4. Ablaufdiagramme (UML)
5. Vom Diagramm zum Python-Code

Lernziel: Eigene Veggie-Soles-Abläufe erst auf Papier skizzieren, dann sauber in Python übersetzen.

Wie wir heute arbeiten: *Live-Demo* zeigt der Lecturer im Terminal, *Sofort ausprobieren* machen Sie im Notebook 13a.

Wozu Algorithmen modellieren?

Ohne Plan

direkt drauflos coden ...

```
def versand(b, l):  
    if l=="DE" and b<50: return 4.95  
    elif l=="DE": return 0  
    elif b<50: return 9.95  
    else: return 4.95
```

Mit Plan (Pseudocode zuerst)

```
ALGORITHMUS Versand(bestellsumme, land)  
    WENN land = "DE" UND bestellsumme < 50 DANN  
        RÜCKGABE 4.95  
    ...
```

Was ist ein Algorithmus?

- Eine **endliche Folge** präzise definierter Schritte
- Löst ein **klar umrissenes Problem** für beliebige Eingaben
- **Sprachunabhängig** – erst Idee, dann Python/Java/...

Algorithmus vs. Programm

Algorithmus	Programm
Lösungsidee	Konkrete Umsetzung
Sprache: Pseudocode, Diagramm	Sprache: Python, Java, ...
Auf Papier nachvollziehbar	Auf einem Computer ausführbar

Beispiele aus dem Alltag: Kochrezept, Wegbeschreibung, Versandlogik bei Veggie Soles.

Eigenschaften eines guten Algorithmus

Eigenschaft	Bedeutet
Eindeutigkeit	Jeder Schritt ist präzise – keine Interpretation nötig
Endlichkeit	Beschreibung passt auf endlich viele Zeilen
Terminiertheit	Algorithmus endet nach endlich vielen Schritten
Determinismus	Gleiche Eingabe → gleiche Ausgabe
Effektivität	Jeder Schritt ist tatsächlich ausführbar

Diese Eigenschaften sind die **Qualitätskontrolle** – bevor wir Code schreiben.

Pseudocode - Konventionen

Pseudocode	Bedeutung	Python
ALGORITHMUS Name(p1, p2)	Funktion definieren	def name(p1, p2):
← (Pfeil)	Zuweisung	=
WENN ... DANN ... SONST	Bedingung	if / else
SOLANGE ...	Schleife mit Bedingung	while
FÜR i VON 1 BIS n	Zählschleife	for i in range(1, n+1)
RÜCKGABE wert	Wert zurückgeben	return wert

Pseudocode ist **kein Standard** – aber im Studium und in Lehrbüchern nehmen wir diese Konventionen.

Pseudocode-Beispiel: Summe von 1 bis n

Pseudocode

ALGORITHMUS SummeBisN(n)

EINGABE: positive ganze Zahl n

AUSGABE: $1 + 2 + \dots + n$

summe $\leftarrow$ 0

FÜR i VON 1 BIS n:

 summe $\leftarrow$ summe + i

RÜCKGABE summe

Python-Übersetzung

```
def summe_bis_n(n):
```

```
    summe = 0
```

```
    for i in range(1, n + 1):
```

Print-Trace: jeden Schritt sichtbar machen

```
def summe_bis_n(n):  
    summe = 0  
    for i in range(1, n + 1):  
        summe = summe + i  
        print(f"    i={i}, summe={summe}")  
    return summe
```

Ausgabe für n=4:

```
i=1, summe=1  
i=2, summe=3  
i=3, summe=6  
i=4, summe=10
```

Visualisierungsstufe 1: Print-Trace – der einfachste Weg, einem Algorithmus beim Denken zuzuschauen. Wir bauen darauf in 13b/14a/14b auf.

Ablaufdiagramme (UML)

Symbol	Bedeutung
Oval (Start / Ende)	Anfang und Ende des Algorithmus
Rechteck	Aktion / Anweisung
Raute	Verzweigung (Bedingung)
Pfeil	Reihenfolge der Schritte

- Visuelle Sprache, **sprachneutral**
- Ergänzt Pseudocode – besonders bei Verzweigungen
- Lesbar auch für Nicht-Programmierende

Im Notebook 13a sehen Sie ein vollständiges Beispiel: Primzahlprüfung als Ablaufdiagramm.

Vom Diagramm zum Code (Primzahl)

Pseudocode

```
ALGORITHMUS IstPrimzahl(n)
    WENN n < 2 DANN RÜCKGABE FALSCH
    FÜR teiler VON 2 BIS n-1:
        WENN n MOD teiler = 0 DANN
            RÜCKGABE FALSCH
    RÜCKGABE WAHR
```

Python

```
def ist_primzahl(n):
    if n < 2:
        return False
    for teiler in range(2, n):
        if n % teiler == 0:
```

Story-Beispiel: Versandkosten

Veggie Soles berechnet Versandkosten je nach Bestellsumme:

ALGORITHMUS Versand(bestellsumme)

EINGABE: Bestellsumme in EUR

AUSGABE: Versandkosten in EUR

WENN bestellsumme $\geq$ 100 DANN

 RÜCKGABE 0.00

SONST

 RÜCKGABE 4.95

```
def versand(bestellsumme):
```

```
    if bestellsumme  $\geq$  100:
```

```
        return 0.00
```

```
    return 4.95
```

Cheat Card: Pseudocode-Vokabular

Pseudocode	Python
ALGORITHMUS f(x)	def f(x):
EINGABE / AUSGABE	Parameter / return
←	=
WENN c DANN ... SONST ...	if c: ... else: ...
SOLANGE c: ...	while c:
FÜR i VON a BIS b	for i in range(a, b+1)
RÜCKGABE wert	return wert
MOD	%

Faustregel: Erst Pseudocode auf Papier, dann Python im Editor. Spart 80% der Bugs.

Ausblick: 13b - wie *gut* ist mein Algorithmus?

- Zwei Algorithmen, gleiches Ergebnis – wer ist schneller?

Variante A: Schleife -- $O(n)$

```
summe = 0
```

```
for i in range(1, n + 1):  
    summe += i
```

Variante B: Formel -- $O(1)$

```
summe = n * (n + 1) // 2
```

- 13b: **Big-O-Notation** – die universelle Sprache für “schnell” vs. “langsam”
- 14a: drei Sortieralgorithmen, live im Vergleich
- 14b: Suche – linear vs. binär (warum sich Sortieren lohnt)

Heute geübt

- ✓ Algorithmus von Programm unterschieden
- ✓ Eigenschaften eines korrekten Algorithmus benannt
- ✓ Pseudocode geschrieben und gelesen
- ✓ Ablaufdiagramme (UML) interpretiert
- ✓ Pseudocode systematisch in Python übersetzt
- ✓ Print-Trace als erste Visualisierungsstufe genutzt

Zur Vertiefung im Notebook 13a:

- Angeleitete Übungen 1.1 – 6.x in Kap. 1-6
- Abschlussübungen 1-4: Pseudocode↔Python, Diagramm→Code, Listen-Minimum, Pseudocode-Bug finden